

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 3 – FILES AND DIRECTORIES



LECTURE PLAN

- File permissions
- Work with directories
- Upload and download files
- Write and Read data to files
- Open and close a file stream
- Manage files and directories

FILE PERMISSIONS

- **File permissions** determine the actions that a specific user can and cannot perform on a file
- Files and directories have three levels of access:
 - User
 - Group
 - Other
- The three typical permissions for files and directories are:
 - Read (r)
 - Write (w)
 - Execute (x)

FILE PERMISSIONS

- In PHP, file permissions are calculated using a four-digit octal (base 8) value

Permissions	First Digit (Leftmost) Always 0	Second Digit User (u)	Third Digit Group (g)	Fourth Digit (Rightmost) Other (o)
Read (r)	0	4	4	4
Write (w)	0	2	2	2
Execute (x)	0	1	1	1

PHP Programming with MySQL, 2011, Cengage Learning.

- The first digit is always 0
- To assign more than one value to an access level, add the values of the permissions together

FILE PERMISSIONS

- `chmod()` - changes the permissions or modes of a file or directory

```
chmod($filename, $mode) //0754
```

- `fileperms()` - reads the permissions associated with a file

```
$perms = fileperms($testfile);  
$perms = decoct($perms % 01000);  
echo "file permissions for $testfile: 0" . $perms . "<br>";
```

READING DIRECTORIES

- The following table lists the PHP functions for working with directories

Function	Description
<code>chdir(<i>directory</i>)</code>	Changes to the specified directory
<code>chroot(<i>directory</i>)</code>	Changes the root directory of the current process to the specified directory
<code>closedir(<i>handle</i>)</code>	Closes a directory handle
<code>getcwd()</code>	Gets the current working directory
<code>opendir(<i>directory</i>)</code>	Opens a handle to the specified directory
<code>readdir(<i>handle</i>)</code>	Reads a file or directory name from the specified directory handle
<code>rewinddir(<i>handle</i>)</code>	Resets the directory pointer to the beginning of the directory
<code>scandir(<i>directory</i>[, <i>sort</i>])</code>	Returns an indexed array containing the names of files and directories in the specified directory

PHP Programming with MySQL, 2011, Cengage Learning

READING DIRECTORIES

- `opendir()` - opens a directory and iterate through entries in a directory
 - A **handle** is a special type of variable that PHP used to represent a resource such as a file or a directory
 - The **directory pointer** is a special type of variable that refers to the currently selected record in a directory listing
- `readdir()` - returns the current file or directory name and move the directory pointer to the next

READING DIRECTORIES

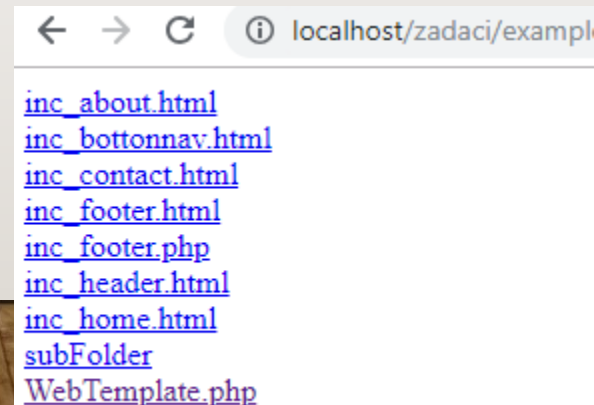
- The PHP scripting engine returns the navigation shortcuts (“.” for current directory, and “..” for parent directory) when it reads a directory
 - `strcmp()` can be used to exclude those entries

READING DIRECTORIES – EXAMPLE

```
<?php
$Dir = ".";    //$Dir = "name_of_dir";
$DirOpen = opendir($Dir);
while ($CurFile = readdir($DirOpen)) {
    if ((strcmp($CurFile, '.') != 0) && (strcmp($CurFile, '..') != 0))

        echo "<a href=\"./\" . $CurFile .\">\" . $CurFile . "</a><br/>\n";

    // echo "<a href=\"name_of_dir/\" . $CurFile . \">\" . $CurFile .
        "</a><br />\n";
}
closedir($DirOpen);
?>
```



READING DIRECTORIES – EXAMPLE

- `scandir()` - returns the names of the entries in a directory to an array sorted in ascending alphabetical order (if we pass 1 as second argument the entries will be sorted in descending alphabetical order)

...

```
$Dir = ".";
$DirEntries = scandir($Dir);    // scandir($Dir, 1);
foreach ($DirEntries as $Entry) {
    if ((strcmp($Entry, '.') != 0) && (strcmp($Entry, '..') != 0))
        echo "<a href=\"./\" . $Entry . "\">" . $Entry .
                                                    "</a><br />\n";
}
```

← → ↻ ⓘ localh

[FileDownloader.php](#)
[PHPCodeBlocks.php](#)
[ViewFiles.php](#)
[backup](#)
[comments](#)
[createDatabase.php](#)
[createTable.php](#)
[example-ch10](#)
[example-ch9](#)
[example-pg102.php](#)
[example-pg104.php](#)
[example-pg106.php](#)
[example-pg119](#)
[example-pg126.php](#)
[example-pg127.php](#)
[example-pg133.php](#)
[example-pg135.php](#)
[example-pg139.php](#)
[example-pg140.php](#)
[example-pg142.php](#)
[example-pg148.php](#)
[example-pg149.php](#)
[example-pg152.php](#)
[example-pg158.php](#)
[example-pg17.php](#)
[example-pg189.php](#)
[example-pg191.php](#)

CREATING DIRECTORIES

- `mkdir()` - creates a new directory

```
mkdir("volunteers");
```

- Use a relative or an absolute path

```
mkdir("../event");
```

```
mkdir("/bin/PHP/utilities");
```

- Other PHP functions to work with directories and files

UPLOADING FILES

- To upload file, web form can be used with “post” method and `enctype` attribute with value “multipart/form-data”
- The `file` input field creates a Browse button for the user to navigate to the appropriate file to upload

```
<input type="file" name="filefield" />
```

- To limit the file size, `MAX_FILE_SIZE` (uppercase) attribute of a hidden input field can be used
 - the `MAX_FILE_SIZE` hidden field must appear before the file input field

FILE INFORMATION

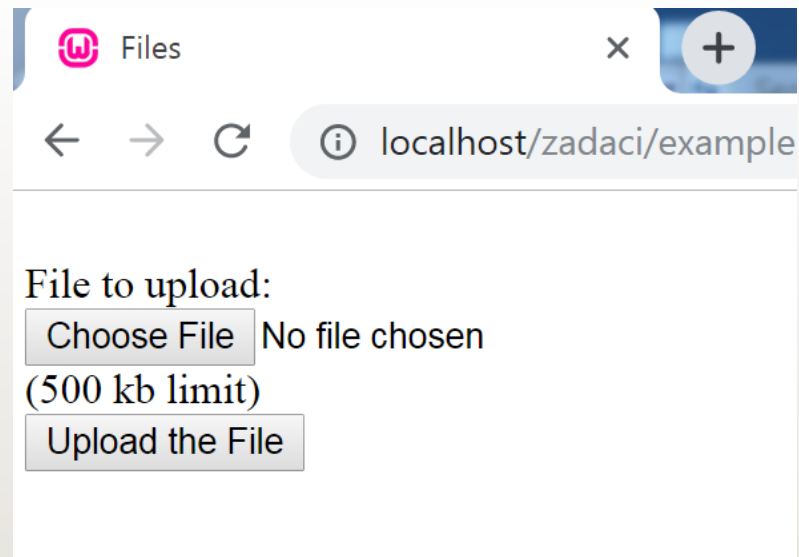
- When the form is posted, information for the uploaded file is stored in the `$_FILES` autoglobal array
- The `$_FILES[]` array element contains five elements:
 - `$_FILES['filefield']['error']` - contains the error code associated with the file
 - `$_FILES['filefield']['tmp_name']` - contains the temporary location of the file contents
 - `$_FILES['filefield']['name']` - contains the name of the original file
 - `$_FILES['filefield']['size']` - contains the size of the uploaded file in bytes
 - `$_FILES['filefield']['type']` - contains the type of the file

UPLOADING FILES

- When uploading files there are some considerations to be taken into account
 - size of the file
 - type of the file
 - security considerations
 - is the file public (freely available to anyone visiting the Web site) or private (only available to authorized visitors)
- `move_uploaded_file()` - moves the uploaded file from its temporary location to a permanent destination

```
move_uploaded_file($filename, $destination)
```

UPLOADING FILES - EXAMPLE



DOWNLOADING FILES

- Files in the public HTML directory structure can be downloaded with an HTML hyperlink
- Files outside the public HTML directory require a three-step process:
 - Tell the script which file to download (can be used URL tokens)
 - Provide the appropriate headers
 - Send the file
- The headers must be send prior to any web content
- `header ()` - can be used to return header information to the Web browser

CONTENT HEADERS FOR DOWNLOADING A FILE

Header	Description	Value	Example
Content-Description	Description of the message contents	A text message	<code>header("Content-Description: File Transfer");</code>
Content-Type	MIME type and subtype of the message contents	A MIME type/subtype string	<code>header("Content-Type: application/force-download");</code>
Content-Disposition	The attributes of the attachment, especially the filename	A series of name/value pairs defining the attributes of the file	<code>header("Content-Disposition: attachment; filename=\"list.txt\"");</code>
Content-Transfer-Encoding	The method used to encode the message contents	7bit, 8bit, quoted-printable, base64, binary	<code>header("Content-Transfer-Encoding: base64");</code>
Content-Length	The length of the message contents	Number	<code>header("Content-Length: 5000");</code>

PHP Programming with MySQL, 2011, Cengage Learning.

DOWNLOADING FILES - EXAMPLE

WRITING CONTENT INTO FILE

- `file_put_contents()` - writes or appends a text string to a file and returns the number of bytes written to the file

`file_put_contents (filename, string[, options])`

- returns number of bytes written into the file
- if `FILE_APPEND` constant is sent as argument, the data will be appended to any existing contents in the specified file instead of overwriting it

READING CONTENT FROM FILE

- `file_get_contents()` - reads the entire contents of a file

```
$myfile = file_get_contents("my_file.txt");  
echo $ myfile;
```

- `readfile()` - displays the contents of a text file to a Web browser

```
readfile("my_file.txt");
```

- `file()` - reads the entire contents of a file into an indexed array
 - Automatically recognizes whether the lines in a text file end in `\n`, `\r`, or `\r\n`, to assign the lines into the elements in the array

WRITING/READING FILE - EXAMPLE

Event Volunteers

To sign up to volunteer at the event, enter your first and last name and click the Registerbutton.

First Name:

Last Name:

Register

Volunteers

Gjorgievska, Elena Vlahu, Elena Brown, Tom

Gjorgievska, Elena Vlahu, Elena Brown, Tom

- 1.Gjorgievska, Elena
- 2.Vlahu, Elena
- 3.Brown, Tom

OPENING AND CLOSING FILE STREAMS

- A **stream** is a channel used for accessing a resource that you can read from and write to
- The **input stream** reads data from a resource (such as a file)
- The **output stream** writes data to a resource
 1. Open the file stream with the `fopen()` function
 2. Write data to or read data from the file stream
 3. Close the file stream with the `fclose()` function

OPENING AND CLOSING FILE STREAMS

- A **handle** is a special type of variable that PHP uses to represent a resource such as a file
- A **file pointer** is a special type of variable that refers to the currently selected line or character in a file

- `fopen()` - opens a handle to a file stream

```
open_file = fopen("text file", "method");
```

- `fclose` - closes file stream

```
fclose($handle);
```


OPENING AND CLOSING FILE STREAMS

Argument	Description
a	Opens the specified file for writing only and places the file pointer at the end of the file; attempts to create the file if it doesn't exist
a+	Opens the specified file for reading and writing and places the file pointer at the end of the file; attempts to create the file if it doesn't exist
r	Opens the specified file for reading only and places the file pointer at the beginning of the file
r+	Opens the specified file for reading and writing and places the file pointer at the beginning of the file
w	Opens the specified file for writing only and deletes any existing content in the file; attempts to create the file if it doesn't exist
w+	Opens the specified file for reading and writing and deletes any existing content in the file; attempts to create the file if it doesn't exist
x	Creates and opens the specified file for writing only; returns FALSE if the file already exists
x+	Creates and opens the specified file for reading and writing; returns FALSE if the file already exists

WRITING/READING USING STREAM

- `fwrite()` - incrementally writes data to a text file
`fwrite($handle, data[, length])`
 - returns the number of bytes that were written to the file
- `fgets()` - reads a single line from a text file (using the file pointer to iterate through the file)
- `fgetc()` - reads a single character from a text file
- `fread()` - reads a number of characters (specified length) from a text file
- `feof()` - returns a value of TRUE when a file pointer reaches the end of a file
 - it accepts the handle for the open file as a single argument

WRITING/READING USING STREAM

Function	Description
<code>fgetc(\$handle)</code>	Returns a single character and moves the file pointer to the next character
<code>fgetcsv(\$handle, length[, delimiter, string_enclosure])</code>	Returns a line, parses the line for CSV fields, and then moves the file pointer to the next line
<code>fgets(\$handle[, length])</code>	Returns a line and moves the file pointer to the next line
<code>fgetss(\$handle, length[, allowed_tags])</code>	Returns a line, strips any XHTML tags the line contains, and then moves the file pointer to the next line
<code>fread(\$handle, length)</code>	Returns up to <i>length</i> characters and moves the file pointer to the next available character
<code>stream_get_line(\$handle, length, delimiter)</code>	Returns a line that ends with a specified delimiter and moves the file pointer to the next line

WRITING/READING USING STREAM EXAMPLE

- Visitors feedback

MANAGING FILES AND DIRECTORIES

- PHP can be used to manage files and the directories that store them
- Among the file directory and management tasks for files and directories are
 - Copying
 - Moving
 - Renaming
 - Deleting

COPYING AND MOVING FILES

- `copy()` – creates a copy of a file
`copy(source, destination)`
- *source* and *destination* arguments:
 - include just the name of a file to make a copy in the current directory
 - specify the entire path for each argument

RENAMING FILES AND DIRECTORIES

- `rename()` - renames a file or directory with PHP

`rename(old_name, new_name)`

REMOVING FILES AND DIRECTORIES

- `unlink()` - to deletes files
- `rmdir()` - deletes directories (the directory must be empty)
- `file_exists()` – returns true/false if a file or directory name exists

